

Seven Seas A-15

Security Audit

Aug 15, 2024

Version 1.0.0

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Seven Seas's smart contract code as found in the section titled 'Source Code'. The security audit was performed by the Macro security team from July 31st to Aug 2nd 2024 and from Aug 12th to Aug 15th 2024.

The purpose of this audit is to review the source code of certain Seven Seas Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Code Quality	2	-	-	2
Informational	1	-	-	-

Seven Seas was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Seven Seas team.
- Available documentation in the repository.

Source Code

The following source code was reviewed during the audit:

- **Repository:** [boring-vault](#)
- **Commit Hash:** `e12c1e320cca1169c9785a8c7d424c38fca8d688`

Specifically, we audited the changes for the following contracts in [PR 61](#).

Source Code	SHA256
<code>src/base/DecodersAndSanitizers/EtherFiLiquidityEthDecoderAndSanitizer.sol</code>	<code>d6ab0f6fef5a120f744d0b165e06d9aa45ba8839e365a1418a9f86587f35f5ed</code>
<code>src/base/DecodersAndSanitizers/Protocols/TellerDecoderAndSanitizer.sol</code>	<code>ef400712564fa25698d38c9439ee6755827417dd88e51bd7f8b6a103d201fe85</code>

We also audited the changes for the following contract in [PR 62](#).

Source Code	SHA256
<code>src/base/DecodersAndSanitizers/Protocols/PendleRouterDecoderAndSanitizer.sol</code>	<code>0cb6cbfcfb9be77b5593ef344870ae4303d003676afa51ab4ae4bb23a14918f0</code>

We also audited the new functionality added for the same contract in [PR 70](#).

Source Code	SHA256
<code>src/base/DecodersAndSanitizers/Protocols/PendleRouterDecoderAndSanitizer.sol</code>	<code>d07ccddf90e42d9807ec27b1e52ce018445a7df569f433e08433741267f37793</code>

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain

to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

- [Q-1](#) Unnecessary encoding of empty `addressesFound`
- [Q-2](#) Project setup should list all required environment variables
- [I-1](#) Inadequate authorization may allow strategist to create unbacked shares

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:

- How bad things can get (for a vulnerability)
- The significance of an improvement (for a code quality issue)
- The amount of gas saved (for a gas optimization)

2. The high/medium/low **likelihood** of the issue:

- How likely is the issue to occur (for a vulnerability)

3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

Q-1

Unnecessary encoding of empty addressesFound

TOPIC	STATUS	QUALITY IMPACT
Best practices	Fixed ↗	Low

In the `PendleRouterDecoderAndSanitizer`, `_sanitizeLimitOrderData()` contains code that tries to concatenate the `limitRouter` address value and named result variable `addressesFound`. At the place of invocation, `addressesFound` is not initialized, so `encodePacked()` call results in the same value as if `abi.encodePacked(limit.limitRouter)` was called.

```
function _sanitizeLimitOrderData(DecoderCustomTypes.LimitOrderData calldata limitOrderData)
    internal
    pure
    virtual
    returns (bytes memory addressesFound)
{
    if (limit.limitRouter != address(0)) {
        // Trying to fill limit orders.
        addressesFound = abi.encodePacked(addressesFound, limit.limitRouter);
        ...
    }
}
```

Consider changing line above to a more readable implementation with less gas overhead.

Q-2

Project setup should list all required environment variables

TOPIC	STATUS	QUALITY IMPACT
Best practices	Fixed ↗	Low

Currently, the project readme provides project setup instructions. However, some of the instructions are not up to date, and some information is missing. For example, the sample.env file, recommended to bootstrap .env secrets file, does not contain all required variables for the project test suite to pass without reverts due to missing environment variables.

For example, in addition to `MAINNET_RPC_URL` , which is already defined, `ARBITRUM_RPC_URL` and `BASE_RPC_URL` are also required.

Consider updating README and sample.env with up-to-date project setup instructions.

I-1

Inadequate authorization may allow strategist to create unbacked shares

TOPIC	IMPACT
Edge case	Informational *

With additional new functionality introduced with `TellerDecoderAndSanitizer` contract, strategist may invoke `bulkDeposit()` and `bulkWithdraw()` on instances of `TellerWithMultiAssetSupport` contract which act as a gatekeepers for corresponding instances of `BoringVault` .

When this functionality is invoked on instances of Teller which are associated with different Vault that the one triggering the call everything works as expected. However, when it is allowed for the calling contract `BoringVault` to invoke `bulkDeposit()` on Teller instance associated with itself, it can reenter the contract and mint extra shares without any actual asset transfer.

Consider scenario:

1. Strategist configures vault to approve itself for max transfer of base asset.
2. Strategist calls `bulkDeposit()` on the vault's teller. When this call is executed it reenters the vault. When `vault.enter()` is called with deposit amount equal to amount of assets in the vault it doubles number of shares without changing underlying asset supply as `from == to`.
3. Strategist may then call `bulkWithdraw()`. When `bulkWithdraw()` is called with amount of shares equal to the half of the Vault's share supply, it receives whole asset balance back as other half is unbacked.
4. As half of the remaining shares are equal to the initial totalSupply of shares, the corresponding check in `ManagerWithMerkleVerification` is passed, even though there are no more assets in the vault.

```
if (totalSupply != vault.totalSupply()) {  
    revert ManagerWithMerkleVerification__TotalSupplyMustRemainConstantDur:  
}
```

For a malicious strategist to exploit this edge case, it is necessary for it get authorization to perform `bulkDeposit()` on the same instance of the vault as well as to perform `bulkWithdrawal()` with receiver being under its control. Since these permissions are currently monitored and carefully granted by the project team this is not exploitable.

However, in the future with potential decentralization efforts, end users should take this into consideration when granting permissions to strategists.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Seven Seas team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.